

Ansteuern von *Light-Emitting Diodes* (LEDs) & Integration von Tastern(*Buttons*)

Übersicht

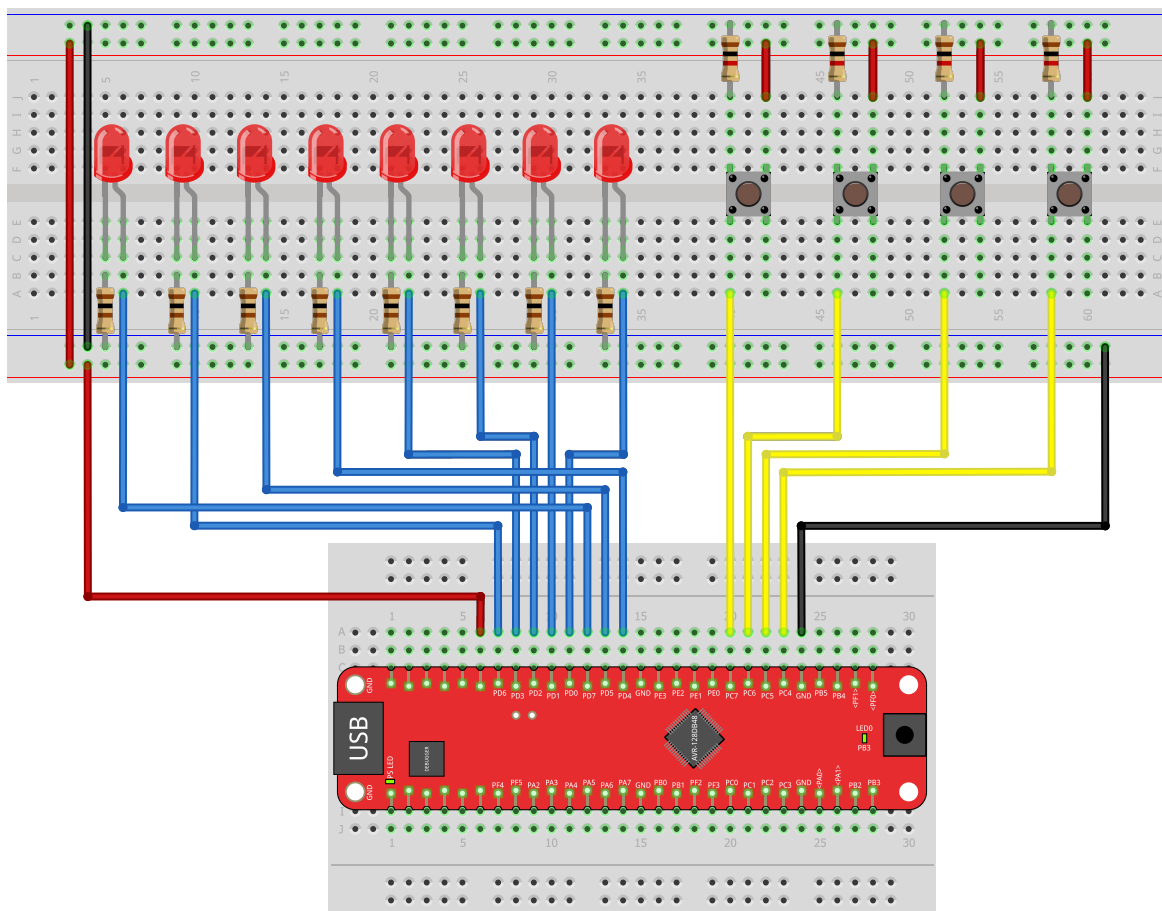
LEDs	2
<i>Light Up (0.5 Punkte)</i>	3
<i>Blinky (2 Punkte)</i>	3
<i>Moving Light (4 Punkte)</i>	3
<i>Binary Counter (4 Punkte)</i>	4
Taster	5
<i>Light By Button (0.5 Punkte)</i>	5
<i>Debounced Counter (4.5 Punkte)</i>	5
<i>Binary Calculator I (4.5 Punkte)</i>	5

LEDs

Bei Ihrer dritten Übung dreht sich alles um das Ein- und Ausschalten von Light-Emitting Diodes, sogenannten LEDs. Wie in [Versuchsaufbau 1](#) zu sehen, benötigen Sie für die folgenden Übungsaufgaben acht LEDs und Widerstände, welche als Vorwiderstände in einer Reihenschaltung von LED und Widerstand agieren.

Bitte schließen Sie die LEDs mit ihren längeren Beinchen (Anode, +) an die Pins PD0 bis PD7 des Curiosity Nano Boards. Die kürzeren Beinchen (Kathode, -) werden mithilfe der Vorwiderstände an der Ground-Leitung des Boards befestigt. Nutzen Sie die beiliegenden Kabel, um die unterschiedlichen Pins miteinander zu verbinden.

Bitte überprüfen Sie Ihre Schaltung noch einmal ganz genau (siehe [Versuchsaufbau 1](#), bevor Sie das Board per USB anschließen.



Light Up (0.5 Punkte)

Bringen Sie nur die LED an Pin **PD7** zum leuchten.

- Nutzen Sie keine fest kodierten Werte in ihren Funktionen. (defines erlaubt)
- Nutzen Sie für die Adressierung der Register nur defines aus der „avr/io.h“ Datei.
- Nutzen Sie für das schreiben einzelner Bits in ein Register nur defines aus der „avr/io.h“ Datei.

Blinky (2 Punkte)

Erweitern Sie Ihr erstes Programm so, dass die leuchtende LED nicht mehr konstant leuchtet, sondern in einem gleichmäßigen, zeitlichen Abstand blinkt. Nutzen Sie dazu **keine** bereits vorhandenen Bibliotheksfunktionen.

- Wie können Sie ihre Programmausführung verzögern? - Stichwort: *Busy Waiting*
- Was verbirgt sich hinter dem Keyword „volatile“ ?
- Welches der PORTx-Register eignet sich am besten für das wiederholte Ein- und Ausschalten?

Die Blinkgeschwindigkeit soll durch eine angelegte Definition einfach veränderbar sein.

Nutzen Sie in einer Erweiterung des Programms eine beliebige zusätzliche LED und lassen Sie die beiden abwechselnd blinken.

Moving Light (4 Punkte)

Erstellen Sie ein Lauflicht, indem Sie alle LEDs auf dem Versuchsaufbau nacheinander ein- und wieder ausschalten. Lassen Sie dabei stets nur eine einzige LED leuchten.

Das Licht soll sich von links nach rechts sowie danach wieder von rechts nach links bewegen. Lassen Sie diese Animation durchgehend laufen.

- Gibt es einen eleganten Weg, mittels einer Zahl die LEDs wie gewollt einzeln aufleuchten zu lassen?

Variieren Sie auch hier die Geschwindigkeit Ihres Lauflichts mittels einer einfach änderbaren Definition.

Binary Counter (4 Punkte)

Bringen Sie alle LEDs nach einem binären Muster zum leuchten. Lassen Sie die LEDs nach folgendem Schema aufleuchten:

PD7	PD6	PD5	PD4	PD3	PD2	PD1	PD0	Dezimal
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	1
0	0	0	0	0	0	1	0	2
0	0	0	0	0	0	1	1	3
0	0	0	0	0	1	0	0	4
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
1	1	1	1	1	1	1	1	255

Die Zeit zwischen zwei angezeigten Zahlen soll variabel sein.

- Was versteht man unter dem sogenannten arithmetischen Überlauf, und ist dieser hier von Nutzen?

Taster

Bei den folgenden Aufgaben werden die 4 Buttons des Versuchsaufbaus verwendet. Die Buttons werden, wie in [Versuchsaufbau 1](#) zu sehen, mit der einen Seite ihrer Anschlusspins an die Pins PC4 bis PC7 angeschlossen und mit der anderen Seite an die 3.3V-Leitung und an die Ground-Leitung des Boards.

Bitte fügen Sie bei der Schaltung zusätzlich die sogenannten *Pull-Down-Widerstände* zwischen den Ground-Leitungen und den Pins der Buttons ein, so, wie bei [Versuchsaufbau 1](#) zu sehen. \\ Solange keiner der Buttons gedrückt wird, sind alle Buttons über die Pull-Down-Widerstände mit Ground verbunden. Erst wenn man einen der Taster drückt, wird auf die nebenliegende Verbindung umgeschaltet und das 3.3 Volt Signal wird an den jeweiligen Pin (PC4 - PC7) weitergeleitet.

Light By Button (0.5 Punkte)

- Lassen Sie die LED, die an PD0 angeschlossen ist, aufleuchten, solange der Button, der an PC4 angeschlossen ist, gedrückt wird.
- Lassen Sie nach dem gleichen Prinzip *alle* LEDs aufleuchten, sobald *irgendein* Button gedrückt wird.

Debounced Counter (4.5 Punkte)

Schreiben Sie ein Programm, welches beim Drücken des Buttons an PC4 eine Zähler-Variable inkrementiert, also hochsetzt. Lassen Sie Ihre Zahl wieder binär kodiert mithilfe der LEDs anzeigen.

Drücken Sie langsam den Taster und lassen Sie langsam wieder los.

- Was können Sie beobachten? Falls Sie nichts erkennen können:
- Was versteht man unter dem Begriff „Button-Bouncing“?

Verändern Sie Ihren Code so, dass das Problem des Tastenprellens behoben wird.

Binary Calculator I (4.5 Punkte)

Nutzen Sie Ihr Wissen aus den vorangegangenen Aufgaben und implementieren Sie einen binären Taschenrechner, welcher die Operationen $+1$, -1 , $\times 2$ und $:2$ beherrscht. Benutzen Sie für jede dieser Operationen einen der vier Buttons und lassen Sie sich das Ergebnis binär codiert auf den LEDs anzeigen.

- Nutzen Sie die effizienteste Methode, eine Zahl mit 2 zu multiplizieren bzw. durch 2 zu dividieren. (Stichwort: Bitoperationen)
- Implementieren Sie einen Zustandsautomaten, sodass immer nur *eine* Operation ausgeführt wird, sollte einer der Knöpfe gedrückt werden. Genauer:
- Wird ein Knopf gedrückt gehalten, wird dessen Operation nicht mehrfach ausgeführt.
- Wird ein Knopf gedrückt, während bereits ein anderer gedrückt gehalten wird, wird dieser ignoriert.

